BLoC (Business Logic Component) pattern is separation of concerns. The app is divided into three main layers:

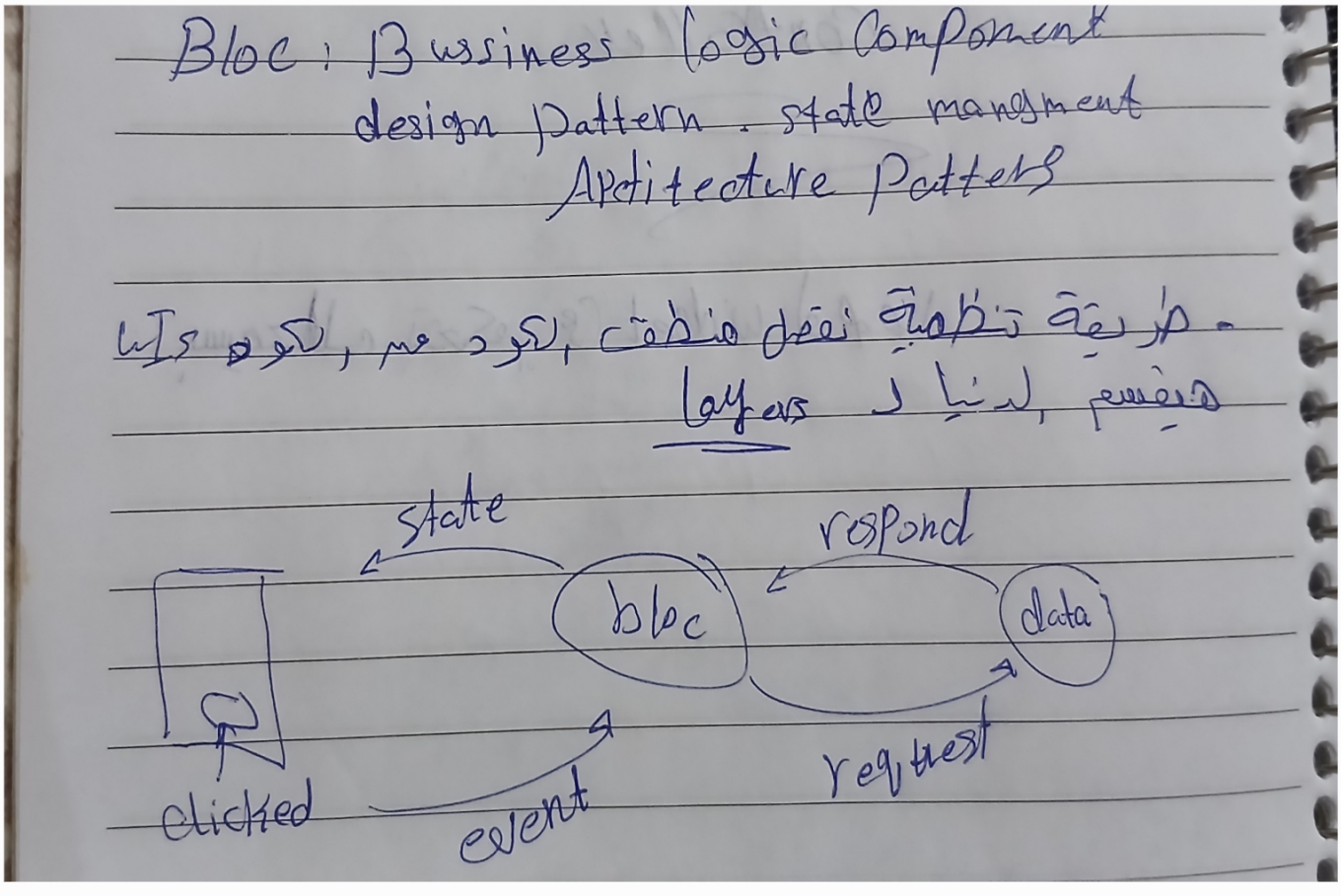
Data Layer:
Responsible for handling data. It includes:
Models (data structure)
Web Services (API calls)
Repository (connects data sources together)

Business Logic Layer:
This is where Cubit or BLoC exists.
It acts as a middle layer between the UI and the data.
It receives input and outputs states.

Presentation Layer (UI):
Displays data to the user and reacts to state changes.

Event:
Represents an action triggered by the user.

State:
Represents the UI condition at a specific moment

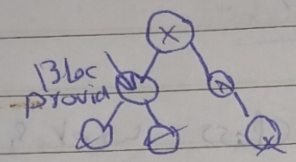


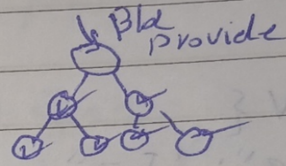
Cubit:
Simpler and easier to use
Does not use events
Directly emits states
Suitable for small or simple features

BLoC:
More structured and scalable
Uses both Events and States
Better for large applications

رقـــم :
التاريخ :

- BlocProvider → bloc توفر widget
بتديك bloc لكل ال widget تحتها في ال tree





By defult, Bloc provided make the code lazy
* UI gotta call it BlocProvider.of <-->(Context)

- BlocBuilder → بتقول للـ bloc provider ابعتلي bloc
- rebuild itself w each state

- Bloclistner → بستخدمه في ال حاجات اللي بتتعمل مره واحده مع ال state change

Bloc consumor → BlocBuilder + Bloclistner

Dio:
A powerful library used for making API requests.

JSON to Dart Models:
Converts JSON responses into Dart objects.

Mapping:
Used to convert a list of JSON objects into a list of models using fromJson.

**Instead of random navigation between screens:
We use a centralized AppRouter
Define routes using onGenerateRoute
This makes navigation more organized and controlled